

documan name ElevatorSubsystem.java

```
leaderMotorPosition = leaderMotor.getPosition();  
    followerMotorPosition = followerMotor.getPosition();  
    simulationEncoder = 0;  
    resetEncoders();
```

```
public class ElevatorSubsystem extends SubsystemBase {  
    public TalonFX leaderMotor;  
    public TalonFX followerMotor;  
    public double simulationEncoder;  
    public StatusSignal<Angle> leaderMotorPosition;  
    private StatusSignal<Angle> followerMotorPosition;
```

```
    public void manualControl(double speed) {  
        leaderMotor.set(speed);  
        followerMotor.set(speed);  
  
        double maxRPM = 6000;  
        double loopTimeSeconds = 0.02;
```

```
double revolutionsPerSecond = (maxRPM / 60.0) * Math.abs(speed);  
  
double revolutionsPerLoop = revolutionsPerSecond * loopTimeSeconds;  
  
simulationEncoder += speed > 0 ? revolutionsPerLoop :  
-revolutionsPerLoop;
```

```
public void OcalPID(double speed, double setpoint) {  
    if (EnabledParts.IS_ELEVATOR_ENABLED) {  
        if (!OI.IS_PID_ENDED) {  
            double leaderPosition = getLeaderMotorEncoder();  
  
            if (isWithinTolerance(leaderPosition,  
Elevator.ELEVATOR_END_VALUE)  
                || isWithinTolerance(leaderPosition, setpoint)) {  
                stopMotors();  
  
                OI.IS_PID_ENDED = true;  
            }  
        }  
    }  
}
```

```
double maxRPM = 6000;  
  
double loopTimeSeconds = 0.02;  
  
double revolutionsPerSecond = (maxRPM / 60.0) *  
Math.abs(speed);
```

```

        double revolutionsPerLoop = revolutionsPerSecond *
loopTimeSeconds;

        if (leaderPosition > setpoint) {
            if (setpoint <= 1) {
                leaderMotor.set(-0.1);
                followerMotor.set(-0.1);
                simulationEncoder -= 0.1 * revolutionsPerLoop / 0.7;
            } else {
                leaderMotor.set(-speed * 0.7);
                followerMotor.set(-speed * 0.7);
                simulationEncoder -= revolutionsPerLoop * 0.7;
            }
        } else {
            leaderMotor.set(speed);
            followerMotor.set(speed);
            simulationEncoder += revolutionsPerLoop;
        }
    }
} else {
    Elevator.ELEVATOR_SPEED = 0.2;
}

OI.IS_PID_ENDED = true;
}

```

```

    }    if (index <= 3) {

        Elevator.PROCESS_SPEED = Elevator.CURRENT_DIRECTION

            ? Elevator.UP_SPEED + index * (Elevator.MAX_UP_SPEED / 6)

            : Elevator.DOWN_SPEED + index *
(Elevator.MAX_DOWN_SPEED / 6);

    } else {

        Elevator.PROCESS_SPEED = Elevator.CURRENT_DIRECTION

            ? Elevator.UP_SPEED + (6 - index) * (Elevator.MAX_UP_SPEED
/ 6)

            : Elevator.DOWN_SPEED + (6 - index) *
(Elevator.MAX_DOWN_SPEED / 6);

    }

```

```

if (index <= 3) {

    Elevator.PROCESS_SPEED = Elevator.CURRENT_DIRECTION

        ? Elevator.UP_SPEED + index * (Elevator.MAX_UP_SPEED / 6)

        : Elevator.DOWN_SPEED + index *
(Elevator.MAX_DOWN_SPEED / 6);

    } else {

        Elevator.PROCESS_SPEED = Elevator.CURRENT_DIRECTION

            ? Elevator.UP_SPEED + (6 - index) * (Elevator.MAX_UP_SPEED
/ 6)

```

```
        : Elevator.DOWN_SPEED + (6 - index) *  
(Elevator.MAX_DOWN_SPEED / 6);  
    }
```

documan name IntakeMoverSubsystem.java

```
public class IntakeMoverSubsystem extends SubsystemBase {  
    public TalonFX leaderMotor;  
    private StatusSignal<Angle> leaderMotorPosition;  
    public RobotContainer container;  
    public double simEncoder;  
  
    public IntakeMoverSubsystem(RobotContainer container) {  
        leaderMotor = new TalonFX(IntakeMover.IM_LEADER_MOTOR_PORT,  
OI.RIO_CANBUS_STRING);  
        leaderMotorPosition = leaderMotor.getPosition();  
        simEncoder = 0;  
        this.container = container;  
        resetEncoders();  
    }
```

```
    public double getLeaderMotorEncoder() {  
        if (RobotBase.isSimulation()) {
```

```
        return simEncoder;
    } else {
        return leaderMotorPosition.refresh().getValueAsDouble();
    }
}
```

```
public void resetEncoders() {
    leaderMotor.setPosition(0);
}
```

```
public void manualControl(double speed) {
    leaderMotor.set(speed);

    double maxRPM = 6000;
    double loopTimeSeconds = 0.02;

    double revolutionsPerSecond = (maxRPM / 60.0) * Math.abs(speed);
    double revolutionsPerLoop = revolutionsPerSecond * loopTimeSeconds;

    simEncoder += speed > 0 ? revolutionsPerLoop : -revolutionsPerLoop;
}
```

```

// OCALPID

public void MoveIT(double speed, double setpoint) {
    if (EnabledParts.IS_INTAKE_MOVER_ENABLED) {
        if (OI.IS_INTAKE_MOVING) {
            double leaderPosition = getLeaderMotorEncoder();
            if (isWithinTolerance(leaderPosition, setpoint)) {
                stopMotors();
                OI.IS_INTAKE_MOVING = false;
            } else {
                double maxRPM = 6000;
                double loopTimeSeconds = 0.02;
                double revolutionsPerSecond = (maxRPM / 60.0) *
Math.abs(speed);
                double revolutionsPerLoop = revolutionsPerSecond *
loopTimeSeconds;

                if (leaderPosition > setpoint) {
                    leaderMotor.set(-speed);
                    simEncoder -= revolutionsPerLoop;
                } else {
                    leaderMotor.set(speed);
                    simEncoder += revolutionsPerLoop;
                }
            }
        }
    }
}

```

```
    } else {  
        OI.IS_INTAKE_MOVING = false;  
    }  
}  
  
private boolean isWithinTolerance(double value, double target) {  
    return value >= target - IntakeMover.OCAL_PID_TOLERANCE_VALUE  
        && value <= target + IntakeMover.OCAL_PID_TOLERANCE_VALUE;  
}  
  
private void stopMotors() {  
    leaderMotor.stopMotor();  
}
```

documan name IntakeSubsystem.java

```
public IntakeSubsystem(RobotContainer container) {  
    leaderMotor = new TalonFX(Intake.INTAKE_LEADER_MOTOR_PORT,  
OI.RIO_CANBUS_STRING);  
    leaderMotorPosition = leaderMotor.getPosition();  
    this.container = container;  
    resetEncoders();  
}
```

```
public void Intake() {  
    if (!EnabledParts.IS_OBJECT_SENSOR_ENABLED) {  
        if (timer.hasElapsed(Intake.INTAKE_TIME)) {  
            leaderMotor.stopMotor();  
            OI.IS_INTAKING = false;  
        } else {  
            leaderMotor.set(-Intake.INTAKE_SPEED);  
        }  
    } else {  
        if (!container.objectDetector.CheckObject()) {  
            leaderMotor.set(-Intake.INTAKE_SPEED);  
        } else {  
            leaderMotor.stopMotor();  
            OI.IS_INTAKING = false;  
        }  
    }  
}
```

```
public void Shoot() {  
    if (!EnabledParts.IS_OBJECT_SENSOR_ENABLED) {
```

```
        if (timer.hasElapsed(-Intake.OUTTAKE_TIME)) {  
            leaderMotor.stopMotor();  
            OI.IS_INTAKING = false;  
        } else {  
            leaderMotor.set(-Intake.OUTTAKE_SPEED);  
        }  
    } else {  
        if (container.objectDetector.CheckObject()) {  
            leaderMotor.set(-Intake.OUTTAKE_SPEED);  
        } else {  
            leaderMotor.stopMotor();  
            OI.IS_INTAKING = false;  
        }  
    }  
}  
  
}
```

I DIDN'T UNDERSTAND THIS CLASSOR
CODES

documan name RotarySwitchSubsystem.java

```
public RotarySwitchSubsystem() {  
    dtPin = new DigitalInput(RotarySwitch.DT_CHANNEL);  
    clkPin = new DigitalInput(RotarySwitch.CLK_CHANNEL);  
}
```

```
        swPin = new DigitalInput(RotarySwitch.SW_CHANNEL);

        lastClockState = clkPin.get() ? 1 : 0;

        totalTurns = 0;
    }

    @Override
    public void periodic() {

        int clkState = clkPin.get() ? 1 : 0;

        int dtState = dtPin.get() ? 1 : 0;

        if (clkState != lastClockState) {
            if (dtState != clkState) {
                totalTurns++;
            } else {
                totalTurns--;
            }
            lastClockState = clkState;
        }

        SmartDashboard.putNumber("RotarySwitch", totalTurns);
    }

    public int getTotalTurns() {
        return totalTurns;
    }
}
```

```
public boolean isSwitchPressed() {  
    return !swPin.get();  
}  
}
```

documan name **CONTANTS**

```
package frc.robot;
```

```
import com.pathplanner.lib.config.PIDConstants;
```

```
import edu.wpi.first.math.geometry.Translation3d;
```

```
import edu.wpi.first.math.util.Units;
```

```
import edu.wpi.first.wpilibj.LEDPattern;
```

```
import edu.wpi.first.wpilibj.LEDPattern.GradientType;
```

```
import edu.wpi.first.wpilibj.util.Color;
```

```
import edu.wpi.first.wpilibj2.command.button.CommandPS5Controller;
```

```
import edu.wpi.first.wpilibj2.command.button.CommandXboxController;
```

```
import edu.wpi.first.wpilibj2.command.button.Trigger;
```

```
import swervelib.math.Matter;
```

```
public final class Constants {  
  
    public static final double ROBOT_MASS = 104.72 * 0.453592;  
  
    public static final Matter CHASSIS = new Matter(new Translation3d(0, 0,  
Units.inchesToMeters(8)), ROBOT_MASS);  
  
    public static final double LOOP_TIME = 0.13; // s, 20ms + 110ms sprk max  
velocity lag  
  
    public static final double MAX_SPEED = 3;  
  
    public static final double LIMELIGHT_KP = 0.2;  
  
    public static final double LIMELIGHT_TX = 12;  
  
    public static final double LIMELIGHT_RIGHT_TIME = 0.09;  
  
    public static final double LIMELIGHT_ALGEA_TIME = 0.76;  
  
    public static final double LIMELIGHT_RIGHT_BRANCH_TIME = 1.6533169;  
  
  
    public static Color setBrightness(Color currentColor, double brightness) {  
        return new Color(currentColor.red * brightness, currentColor.green *  
brightness,  
        currentColor.blue * brightness);  
    }  
}
```

```
public static final class DrivebaseConstants {  
    public static final double WHEEL_LOCK_TIME = 10;  
    public static final PIDConstants transation = new PIDConstants(12, 0.0, 0.05);  
    public static final PIDConstants angle = new PIDConstants(3, 0.0, 0.05);  
}
```

```
public static class OperatorConstants {  
    public static final double DEADBAND = 0.1;  
    public static final double LEFT_Y_DEADBAND = 0.1;  
    public static final double RIGHT_X_DEADBAND = 0.1;  
    public static final double TURN_CONSTANT = 6;  
}
```

```
public static final class LedSubsystem {  
    public static int LED_PWM_PORT = 0;  
    public static int LED_LENGTH = 180;  
    public static int BREATHE_MAGNITUDE = 5;  
    public static LEDPattern ELEVATOR_PROCESS_COLOR =  
LEDPattern.solid(setBrightness(Color.kPurple, 0.69 * 0.8));  
    public static LEDPattern TARGET_FOCUS_COLOR =  
LEDPattern.solid(setBrightness(Color.kGreen, 0.69 * 0.8));  
    public static LEDPattern INTAKE_COLOR =  
LEDPattern.solid(setBrightness(Color.kWhite, 0.69 * 0.8));  
    public static LEDPattern ALGEA_INTAKE_COLOR =  
LEDPattern.solid(setBrightness(Color.kAqua, 0.69 * 0.8));  
    public static LEDPattern BREATHE_COLOR =  
LEDPattern.gradient(GradientType.kDiscontinuous,  
        setBrightness(Color.kOrangeRed, 0.69 * 0.8),  
        setBrightness(Color.kRed, 0.69 * 0.8),  
        setBrightness(Color.kDarkRed, 0.69 * 0.8));  
}
```

```
public static final class RotarySwitch {  
    public static int DT_CHANNEL = 3;  
    public static int CLK_CHANNEL = 4;  
    public static int SW_CHANNEL = 5;  
}
```

```
public static final class Elevator {  
    public static int ELEVATOR_LEADER_MOTOR_PORT = 14;
```

```
public static int ELEVATOR_FOLLOWER_MOTOR_PORT = 13;

public static double ELEVATOR_SPEED = 0.25;

public static double ELEVATOR_DOWN_VOLTS = 0.05;

public static double ELEVATOR_STATIC_VOLTS = 0.3;

public static double OCAL_PID_TOLERANCE_VALUE = 0.35; // TODO
Increase If System Doesn't Work

public static double ELEVATOR_START_VALUE = 0; // Generally True

public static double ELEVATOR_END_VALUE = 34.5;

public static double ELEVATOR_SOURCE_VALUE = 0;

public static double ELEVATOR_PROCESSOR_VALUE = 1.5;

public static double ELEVATOR_TOZERO_VALUE = 0;

public static double ELEVATOR_ALGZERO_VALUE = 3;

public static double ELEVATOR_L1_VALUE = 4.15;

public static double ELEVATOR_L2_VALUE = 8.10;

public static double ELEVATOR_L3_VALUE = 20.21;

public static double ELEVATOR_L4_VALUE = 35;

public static double ELEVATOR_ALGEA_VALUE_DOWN = 2.5;

public static double ELEVATOR_ALGEA_VALUE_MIDDLE = 18.3;
```

```
public static double UP_SPEED = 0.3;

    public static double MAX_UP_SPEED = 0.6;

    public static double DOWN_SPEED = 0.12;

    public static double MAX_DOWN_SPEED = 0.18;

    public static boolean CURRENT_DIRECTION = true;

    public static double PROCESS_START_POSITION = 0;

    public static double PROCESS_SPEED = 0.3;


public static class Intake {

    // Intake Motor Port

    public static double INTAKE_REVERSE_SPEED = 0.3;

    public static int INTAKE_LEADER_MOTOR_PORT = 16;
```

```
public static double INTAKE_SPEED = 0.13;

public static double OUTTAKE_SPEED = 0.58;

public static double INTAKE_TIME = 0.5;

public static double OUTTAKE_TIME = 1;

}
```

```
public static class AlgeaIntake {

    // Algea Intake Motor Port

    public static int ALGEA_LEADER_MOTOR_PORT = 16; // Shoud Be Same
With Intake

    public static double INTAKE_SPEED = 0.2;

    public static double OUTTAKE_SPEED = 0.4;

    public static double INTAKE_TIME = 15;

    public static double OUTTAKE_TIME = 1;
```

```
public static double ALGEA_STATIC_VOLTAGES = 0.050;
```

```
public static class IntakeMover {
```

```
    public static int IM_LEADER_MOTOR_PORT = 15;
```

```
    public static double INTAKE_MOVER_SPEED = 0.15;
```

```
    public static double OCAL_PID_TOLERANCE_VALUE = 0.13;
```

```
    public static double IM_STATIC_VOLTAGE = 0.15;
```

```
    public static double IM_ALGEA_STATIC = 0.25;
```

```
    public static double IM_CORAL_STATIC = 0.05;
```

```
    public static double INTAKE_PROCESSOR_VALUE = 25.5;
```

```
    public static double INTAKE_SOURCE_VALUE = 0;
```

```
    public static double INTAKE_ALGEA_VALUE = 17.5;
```

```
    public static double INTAKE_REEFSCAPE_VALUE = 2.5;
```

```
    public static double INTAKE_CORAL_VALUE = 3.7;
```

```
    public static double INTAKE_L4_VALUE = 2.9;
```

```
    public static double INTAKE_L4_2_VALUE = 11;
```

```
}
```

```
public static class ObjectDetector {  
    public static final int SENSOR_DIO_PORT = 2;  
}
```

```
public static class Controls {
```

```
public static CommandPS5Controller DRIVER_CONTROLLER = new
CommandPS5Controller(OI.DRIVER_CONTROLLER_PORT);

public static CommandXboxController OPERATOR_CONTROLLER = new
CommandXboxController(

    OI.OPERATOR_CONTROLLER_PORT);

// For TEU

public static Trigger ELEVATOR_MANUAL_UP =
DRIVER_CONTROLLER.R2();

public static Trigger ELEVATOR_MANUAL_DOWN =
DRIVER_CONTROLLER.L2();

public static Trigger INTAKE_MOVE_UP = DRIVER_CONTROLLER.R1();
public static Trigger INTAKE_MOVE_DOWN = DRIVER_CONTROLLER.L1();


public static Trigger L1 = DRIVER_CONTROLLER.options();
public static Trigger L2 = DRIVER_CONTROLLER.circle();
public static Trigger L3 = DRIVER_CONTROLLER.cross();
public static Trigger L4 = DRIVER_CONTROLLER.triangle();


public static Trigger GET_SOURCE = DRIVER_CONTROLLER.square();
public static Trigger LED_CYCLE = DRIVER_CONTROLLER.button(14);

public static Trigger ELEVATOR_TOZERO =
DRIVER_CONTROLLER.povDown();


public static int SWERVE_RIGHT_X_AXIS = 2;

public static Trigger RESET_GYRO = DRIVER_CONTROLLER.povUp();


public static Trigger LIMELIGHT_FOCUS = DRIVER_CONTROLLER.L3());
```

```

public static Trigger LIMELIGHT_STOP = DRIVER_CONTROLLER.R3();

public static Trigger ALGEA_INTAKE = DRIVER_CONTROLLER.povLeft();
public static Trigger ALGEA_OUTTAKE = DRIVER_CONTROLLER.povRight();
public static Trigger ALGEA_PROCESSOR = DRIVER_CONTROLLER.PS();
public static Trigger ALGEA_REMOVAL = DRIVER_CONTROLLER.create();

// OPERATOR

public static Trigger CORAL_INTAKE = OPERATOR_CONTROLLER.povLeft();
public static Trigger CORAL_GERIAL =
OPERATOR_CONTROLLER.povRight();

public static Trigger ADMIN = OPERATOR_CONTROLLER.button(9);
public static Trigger LIMELIGHT_PREMIUM = OPERATOR_CONTROLLER.y();
public static Trigger ZERO_ALL = OPERATOR_CONTROLLER.x();
}

public static class Test_Controls {

    public static Trigger T_ELEVATOR_MANUAL_UP =
Controls.DRIVER_CONTROLLER.R2();

    public static Trigger T_ELEVATOR_MANUAL_DOWN =
Controls.DRIVER_CONTROLLER.L2();

    public static Trigger T_INTAKE_MOVE_UP =
Controls.DRIVER_CONTROLLER.R1();

    public static Trigger T_INTAKE_MOVE_DOWN =
Controls.DRIVER_CONTROLLER.L1();

    public static Trigger T_ELEVATOR_ZERO =
Controls.DRIVER_CONTROLLER.povDown();

    public static Trigger T_ALGEA_INTAKE =

```

```
Controls.DRIVER_CONTROLLER.povLeft();
```

```
    public static Trigger T_ALGEA_OUTTAKE =  
Controls.DRIVER_CONTROLLER.circle();
```

```
    public static Trigger T_CORAL_INTAKE =  
Controls.DRIVER_CONTROLLER.square();
```

```
    public static Trigger T_CORAL_OUTTAKE =  
Controls.DRIVER_CONTROLLER.povRight();
```

```
    public static Trigger T_LED_CYCLE =  
Controls.DRIVER_CONTROLLER.triangle();
```

```
    public static Trigger T_LED_MORSE =  
Controls.DRIVER_CONTROLLER.options();
```

```
    public static Trigger T_INTAKE_MOVE_L1 =  
Controls.DRIVER_CONTROLLER.povUp();
```

```
}
```

```
public static class EnabledParts {
```

```
    // Hope Not To Use...
```

```
    public static boolean IS_LED_ENABLED = true;
```

```
    public static boolean IS_OBJECT_SENSOR_ENABLED = true;
```

```
    public static boolean IS_ALGEA_INTAKE_ENABLED = true;
```

```
    public static boolean IS_INTAKE_MOVER_ENABLED = true;
```

```
    public static boolean IS_ELEVATOR_ENABLED = true;
```

```
    public static boolean IS_SWERVE_ENABLED = true;
```

```
    public static boolean IS_ROTARY_SWITCH_ENABLED = false;
```

```
    public static boolean IS_INTAKE_ENABLED = true;
```

```
}
```

```
public static final class OI {
```

```
// General Robot Constants

public static int DRIVER_CONTROLLER_PORT = 0;

public static int OPERATOR_CONTROLLER_PORT = 1;

public static String SWERVE_CANBUS_STRING = "rio";

public static String RIO_CANBUS_STRING = "rio";

public static boolean IS_TEST = false;

public static boolean IS_PROCESSING = false;

public static boolean IS_PID_ENDED = false;

public static boolean IS_INTAKE_MOVING = false;

public static boolean IS_SWERVE_FOCUSED = false;

public static boolean IS_INTAKING = false;

public static boolean IS_ALGEA_INTAKING = false;

public static boolean IS_LED_CYCLING = false;

public static boolean IS_LED_MORSE_SHOWING = false;

public static boolean IS_FOCUSED = false;

public static boolean IS_GONE = false;

public static double LIMELIGHT_SPEED = 0.4;

}

}
```

MAYBE THIS IS **THE END** (just maybe)

**THAT WAS 2025 AMAL HAWS FRC
ROBOT'S CODES**

